

Universidad de Ingeniería y Tecnología

Departamento de Ingeniería Mecatrónica



Robótica Autónoma

Profesores: Oscar E. Ramos, Alexander López

Laboratorio 6

SLAM en 2D y 3D

Lima - Perú

2026-1

Laboratorio 6

SLAM en 2D y 3D

Objetivos

- Obtener el mapa bidimensional de un entorno usando SLAM a través del paquete llamado Gmapping y Hector-mapping.
- Obtener el mapa tridimensional de un entorno usando SLAM a través del paquete llamado RTAB-Map.

Equipo y Materiales

Descripción	Cantidad
Computadora con Internet	1

Indicaciones del Laboratorio

1. Desde el segundo laboratorio, todos los laboratorios son evaluados.
2. Se debe leer la guía del laboratorio antes de asistir a cada laboratorio.
3. El laboratorio comienza a más tardar 5 minutos después de la hora de inicio. El laboratorio empieza con la prueba de entrada, la cual se debe desarrollar de manera personal. Esta prueba de entrada cuenta como parte de la nota del laboratorio, la cual evalúa que el alumn@ haya leído la guía de laboratorio. La prueba tiene una duración de 4 minutos.
4. Los alumnos que lleguen 20 minutos después de iniciado el laboratorio son libres de asistir, pero no tendrán nota en dicho laboratorio.
5. El laboratorio se desarrolla en pareja de 2 personas (solo será de 3 personas en caso falte un alumn@ que no tenga pareja).
6. El laboratorio será evaluado durante el desarrollo del laboratorio, una vez terminada cada actividad, se llamará al profesor para evaluar dicha actividad.
7. Está prohibido copiar, en caso de copia, los alumnos involucrados serán presentados al comité de ética de la universidad.

Resumen del laboratorio

Se presenta un resumen de los puntos a tratar en el siguiente laboratorio.

1. Prueba de entrada (3 puntos).
2. SLAM en 2D (3 puntos).
3. SLAM en 3D (3 puntos).
4. Implementación (4 puntos).

1. Paquetes del laboratorio

Para trabajar en este laboratorio, es necesario descargar el repositorio del curso en el espacio de trabajo “catkin_ws”:

```
$ git clone https://github.com/utecrobotics/robautonoma-2026 autlabs
```

En caso no se tenga los paquetes para trabajar con “Turtlebot3”, aquí se indica los comandos puede descargar dichos paquetes:

```
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3.git
```

```
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
```

```
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
```

Nota: En este laboratorio se utilizará el modelo “waffle_pi” del robot “Turtlebot3”, por tanto, se debe asignar a la variable de entorno “TURTLEBOT3_MODEL” el valor de “waffle_pi”. Esta variable fue añadida al archivo “.bashrc” en el laboratorio 1, revisar el archivo para verificar que ha sido definida.

En caso que se esté trabajando con una computadora en la nube, es necesario trabajar con el robot MerakM:

```
$ git clone https://github.com/alexwbots/merakm_gazebo.git
```

2. SLAM 2D

2.1. Cartographer

Cartographer de Google ofrece una solución en tiempo real para el mapeo de interiores mediante una mochila equipada con sensores que genera mapas de cuadrícula 2D con una resolución de 5 cm. El operador del sistema puede ver cómo se crea el mapa mientras camina por un edificio. Los escaneos láser se insertan en un submapa en la posición estimada óptima, que se considera

suficientemente precisa durante cortos periodos de tiempo. La comparación de escaneos se realiza con un submapa reciente, por lo que solo depende de los escaneos recientes, y el error de las estimaciones de pose en el sistema de coordenadas global se acumula.

Para lograr un buen rendimiento con requisitos de hardware modestos, nuestro enfoque SLAM no utiliza un filtro de partículas. Para gestionar la acumulación de errores, realizamos una optimización de pose periódicamente. Cuando un submapa finaliza, es decir, cuando ya no se insertarán nuevos escaneos, participa en la comparación de escaneos para cerrar el bucle. Todos los submapas y escaneos finalizados se consideran automáticamente para el cierre del bucle. Si están lo suficientemente cerca según las estimaciones de pose actuales, un comparador de escaneos intenta encontrar el escaneo en el submapa. Si se encuentra una coincidencia suficientemente buena en una ventana de búsqueda alrededor de la pose estimada actual, se agrega como restricción de cierre de bucle al problema de optimización. Al completar la optimización cada pocos segundos, la experiencia del operador es que los bucles se cierran inmediatamente cuando se vuelve a visitar una ubicación. Esto conlleva la restricción de tiempo real flexible de que la coincidencia de escaneos de cierre de bucle debe ocurrir más rápido que la adición de nuevos escaneos; de lo contrario, se retrasa notablemente. Logramos esto utilizando un enfoque de ramificación y acotación y varias cuadrículas precalculadas por submapa finalizado. Para instalar el paquete de mapeo debe utilizar la siguiente línea de código:

```
$ sudo apt-get install ros-humble-cartographer ros-humble-cartographer-ros
```

2.1. SLAM Toolbox

Este proyecto ofrece la capacidad de realizar prácticamente todas las funciones de cualquier otra biblioteca SLAM disponible, tanto gratuita como de pago, e incluso más. Esto incluye Estos son los aspectos más destacados:

- El SLAM 2D básico para robótica móvil (iniciar, mapear, guardar archivo .pgm) con útiles herramientas integradas como guardar mapas.
- Permite refinar, remapear o continuar mapeando un grafo de poses guardado (serializado) en cualquier momento.
- Mapeo continuo: cargar un grafo de poses guardado y continuar mapeando en un espacio, eliminando al mismo tiempo la información irrelevante de los escaneos recién añadidos.
- Un modo de localización basado en optimización, construido sobre el grafo de poses. Opcionalmente, ejecute el modo de localización sin un mapa previo para el modo de "odometría lidar" con cierres de bucle locales.
- Modos de mapeo síncrono y asíncrono.
- Fusión de mapas cinemáticos (con una técnica de fusión de manipulación de grafos elásticos en desarrollo).

- Optimizadores basados en plugins con un nuevo plugin optimizado basado en Google Ceres.
- Plugin de RVIZ para interactuar con las herramientas.
- Herramientas de manipulación de grafos en RVIZ para manipular nodos y conexiones durante el mapeo.
- Serialización de mapas y almacenamiento de datos sin pérdida.

Para ejecutar en robots en escenarios reales, recomiendo usar el paquete snap: slam-toolbox, que incluye optimizaciones que lo hacen aproximadamente 10 veces más rápido. Necesita la instalación deb/source para las demás herramientas de nivel de desarrollador que no necesitan estar en el robot (plugins de RVIZ, etc.).

Este paquete ha sido probado en el mapeo de edificios a una velocidad de más de 5 veces la velocidad real hasta aproximadamente 30 000 pies cuadrados y a una velocidad de 3 veces la velocidad real hasta aproximadamente 60 000 pies cuadrados. El área más grande (que yo sepa) utilizada fue un edificio de 200 000 pies cuadrados en modo síncrono (es decir, procesando todos los escaneos, independientemente del retraso), y espacios mucho más grandes en modo asíncrono. Para instalarlo necesita usar la siguiente línea de código:

```
$ sudo apt-get install ros-humble-slam-toolbox
```

3. Sensores del Robot Turtlebot3

En esta sección se busca primero comprender cómo se utilizan los sensores en URDF, para posteriormente poder hacer modificaciones en los sensores que tiene por defecto el robot Turtlebot3. En particular, se buscará quitar la cámara RGB y el LiDAR, y añadir una cámara RGBD (Kinect).

3.1. Sensores en URDF y Gazebo

Como se realizó en laboratorios anteriores, se puede correr la simulación del robot usando el archivo de lanzamiento “turtlebot3_world.launch” que se encuentra en el paquete “autlab6”. Una vez ejecutado el comando, se aprecia el mismo entorno que se observó en el laboratorio anterior. Al abrir este archivo de lanzamiento con un editor de texto (configurando para que sea mostrado como de tipo XML, de preferencia), se puede ver que internamente llama al archivo “urdf/turtlebot3.urdf.xacro”. Abrir este último archivo con un editor de texto para su análisis.

3. SLAM en 3D

Utilizando el sensor Kinect es posible realizar un mapeo tridimensional del entorno del robot usando SLAM. En esta sección se utilizará una técnica de RGB-D graph slam llamada RTAB-map (Real-Time Appearance-Based Mapping) que hace uso de la información que provee el sensor Kinect.

3.1. Usando el Robot Turtlebot3 con Kinect

Para poder realizar el escaneo tridimensional, es necesario primero instalar el paquete de ROS llamado “rtabmap-ros” de la siguiente manera:

```
$ sudo apt-get update
```

```
$ sudo apt-get install ros-humble-rtabmap ros-humble-rtabmap-ros ros-humble-rtabmap-viz
```

Luego, iniciar la simulación del robot Turtlebot3 con Kinect de la sección anterior, asegurándose de que se esté publicando las relaciones entre los diversos sistemas de referencia de las partes del robot (en los tópicos “/tf” y “/tf_static”). Para iniciar el mapeo y localización simultáneos en 3D es necesario correr lo siguiente desde otro terminal:

```
$ ros2 launch rtabmap_ros rtabmap.launch rtabmap_args:="--delete_db_on_start" visual_odometry:=false  
odom_topic:=/odom depth_topic:=/camera/depth/image_raw rgb_topic:=/camera/rgb/image_raw  
rviz:=true rtabmapviz:=false
```

Para el robot MerakM:

```
$ ros2 launch rtabmap_launch rtabmap.launch.py rtabmap_args:="--delete_db_on_start"  
odom_topic:=/odom depth_topic:=/depth_camera/rgbd_sensor/depth/image_raw  
rgb_topic:=/camera/camera_sensor/image_raw camera_info_topic:=/camera/camera_sensor/camera_info  
rviz:=true
```

Nota: todo el comando anterior debe estar en una sola línea. Este comando ejecuta RTAB-map en la simulación de Gazebo del robot. En RViz puede ser conveniente desactivar la visualización de los sistemas de referencia (TF) y puede ser útil añadir un elemento “RobotModel” para visualizar al robot.

Para crear el mapa, es necesario mover al robot, de preferencia no tan rápido. El movimiento del robot se realizará mediante comandos remotos introducidos a través del teclado, con el nodo utilizado en laboratorios anteriores (turtlebot3_teleop). Una vez que se termina el mapeo se pueden cerrar los nodos ejecutados (ya que todo este proceso es computacionalmente costoso). El mapa generado se almacena automáticamente en “~/ros/rtabmap.db”. Es importante notar que si se ejecuta nuevamente el comando de mapeo, se sobrescribirá este archivo (por las opciones que se está pasando a “rtabmap”)

